SESIÓN N° 5

Tema: Archivos de Texto, Archivos Binarios y Manejo de Excepciones

I. Objetivo de la Guía

Implementar algoritmos que utilicen almacenamiento permanente mediante flujos de entrada y salida en archivos de texto plano y archivos binarios, controlando adecuadamente las excepciones de lectura y escritura (IOException).

II. Marco Teórico Breve

1. Archivos de Texto vs. Archivos Binarios

Archivos de texto (Text-File): Almacenan datos legibles por humanos (caracteres). Se usan FileWriter / PrintWriter para escribir y FileReader / BufferedReader para leer.
Archivos binarios: Almacenan datos en formato de bytes (no legibles como texto). Se usan FileOutputStream / ObjectOutputStream para escribir (serializar) y FileInputStream / ObjectInputStream para leer (deserializar).
Piensa en los flujos (streams) como tubos de agua encadenados: FileWriter es el tubo que llega directo al archivo, y PrintWriter es un embudo con forma más cómoda que conectas ENCIMA de ese tubo para verter el texto más fácilmente.

2. Manejo de Excepciones (try-catch)

try-catch es una estructura que "intenta" ejecutar un código riesgoso (try) y, si algo falla, "atrapa" el error (catch) en vez de que el programa se detenga bruscamente.
try-with-resources: Forma moderna de abrir recursos (archivos, streams). Java los cierra automáticamente al salir del bloque, incluso si ocurre una excepción.
IOException: Tipo de excepción que se lanza cuando falla la lectura o escritura de un archivo (no existe, sin permisos, disco lleno, etc.).

Errores Comunes al Empezar

ERROR TÍPICO	CÓMO EVITARLO
Usar código que puede lanzar IOException sin un try-catch ni un throws en la firma del método.	Todo código de archivos es "riesgoso" para Java; siempre debe ir dentro de un try-catch, o el método debe declarar throws IOException.
Abrir un archivo para escribir sin cerrarlo (fuera de un try-with-resources).	Usa siempre try-with-resources (try (FileWriter fw = ...)); Java cierra el archivo automáticamente al terminar, incluso si algo falla.
Intentar leer un archivo que todavía no existe, porque nunca se ejecutó el código que lo crea.	Ejecuta primero el Paso que escribe/crea el archivo, antes de ejecutar el que lo lee.

III. Actividades Prácticas Paso a Paso

Parte 1

Desarrollo del Trabajo 1 (Módulo de Registro en Texto Plano)
Crea una clase llamada HistorialLog.java que permita registrar en un archivo de texto cada vez que un estudiante realice un canje en la tienda del proyecto.

Código completo — HistorialLog.java:

```
import java.io.FileWriter;
import java.io.PrintWriter;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class HistorialLog {

    // Método para guardar un registro en archivo de texto (Trabajo 1)
    public static void registrarCanje(String estudiante, String itemCanjeado) {
        // Uso de FileWriter en modo append (true) para no sobrescribir el archivo
        try (PrintWriter pw = new PrintWriter(new FileWriter("historial_canjes.txt", true)))
{
            pw.println("Estudiante: " + estudiante + " | Ítem: " + itemCanjeado);
            System.out.println("Registro de canje guardado exitosamente en el archivo de
texto.");
        } catch (IOException e) {
            System.out.println("Error de E/S al escribir en el archivo de texto: " +
e.getMessage());
        }
    }

    // Método para leer el historial de texto
    public static void leerHistorial() {
        try (BufferedReader br = new BufferedReader(new FileReader("historial_canjes.txt")))
{
            String linea;
            System.out.println("--- HISTORIAL DE CANJES (TEXT-FILE) ---");
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("No se encontró el archivo o hubo un error al leerlo: " +
e.getMessage());
        }
    }
}
```

Explicación línea por línea (puntos clave):

import java.io.FileWriter;
Importa FileWriter, la clase básica para escribir caracteres en un archivo.
import java.io.PrintWriter;
Importa PrintWriter, que envuelve a FileWriter para escribir texto con métodos como println().
import java.io.BufferedReader;
Importa BufferedReader, que envuelve a FileReader para leer línea por línea.
import java.io.FileReader;
Importa FileReader, la clase básica para leer caracteres desde un archivo.
import java.io.IOException;
Importa IOException, la excepción que hay que atrapar en cualquier operación de archivos.

public class HistorialLog {
Declara la clase HistorialLog.
public static void registrarCanje(String estudiante, String itemCanjeado) {
Declara el método static registrarCanje, que recibe el nombre del estudiante y el ítem canjeado.
try (PrintWriter pw = new PrintWriter(new FileWriter("historial_canjes.txt", true))) {
Abre, con try-with-resources, un PrintWriter que envuelve a un FileWriter apuntando a "historial_canjes.txt" en modo append.
pw.println("Estudiante: " + estudiante + " | Ítem: " + itemCanjeado);
Escribe una línea de texto en el archivo con la información del canje.
System.out.println("Registro de canje guardado exitosamente en el archivo de texto.");
Muestra en consola un mensaje confirmando que el registro se guardó.
} catch (IOException e) {
Bloque catch que atrapa IOException si algo falla al escribir (por ejemplo, sin permisos de escritura).
System.out.println("Error de E/S al escribir en el archivo de texto: " + e.getMessage());
Muestra en consola un mensaje de error con el detalle de la excepción.
}
Cierra el bloque catch.
}
Cierra el método registrarCanje (Java cierra el PrintWriter y su FileWriter interno automáticamente, gracias a try-with-resources).
public static void leerHistorial() {
Declara el método static leerHistorial, sin parámetros.
try (BufferedReader br = new BufferedReader(new FileReader("historial_canjes.txt"))) {
Abre, con try-with-resources, un BufferedReader que envuelve a un FileReader apuntando al mismo archivo de texto.
String linea;
Declara la variable linea, donde se irá guardando cada línea leída.
System.out.println("--- HISTORIAL DE CANJES (TEXT-FILE) ---");
Muestra en consola un encabezado antes de mostrar el historial.
while ((linea = br.readLine()) != null) {
Bucle while: lee una línea con br.readLine() y la guarda en linea; continúa mientras no se llegue al final del archivo (readLine() devuelve null).
System.out.println(linea);
Muestra en consola la línea leída.
}
Cierra el bucle while.
} catch (IOException e) {
Bloque catch que atrapa IOException si el archivo no existe o hay un error al leerlo.
System.out.println("No se encontró el archivo o hubo un error al leerlo: " + e.getMessage());
Muestra en consola un mensaje de error con el detalle de la excepción.
}
Cierra el bloque catch.
}
Cierra el método leerHistorial.
}
Cierra la clase HistorialLog.

Parte 2

Desarrollo del Trabajo 2 (Módulo de Respaldos Binarios)
Crea una clase llamada RespaldoBinario.java que permita guardar un respaldo serializado de la configuración del sistema usando flujos binarios.

Código completo — RespaldoBinario.java:

```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;
import java.io.FileInputStream;
import java.io.ObjectInputStream;
import java.io.IOException;

public class RespaldoBinario {

    // Método para guardar un objeto de configuración en archivo binario (Trabajo 2)
    public static void guardarRespaldoConfig(String configuracion) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("config_respaldo.dat"))) {
            oos.writeObject(configuracion);
            System.out.println("Respaldo binario generado correctamente.");
        } catch (IOException e) {
            System.out.println("Error al escribir el archivo binario: " + e.getMessage());
        }
    }

    // Método para leer el respaldo binario
    public static void leerRespaldoConfig() {
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("config_respaldo.dat"))) {
            String configLeida = (String) ois.readObject();
            System.out.println("Configuración recuperada desde el binario: " + configLeida);
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error al leer el archivo binario: " + e.getMessage());
        }
    }
}
```

Explicación línea por línea (puntos clave):

import java.io.FileOutputStream;
Importa FileOutputStream, la clase básica para escribir bytes en un archivo.
import java.io.ObjectOutputStream;
Importa ObjectOutputStream, que envuelve a FileOutputStream para poder escribir objetos completos (serializados).
import java.io.FileInputStream;
Importa FileInputStream, la clase básica para leer bytes desde un archivo.
import java.io.ObjectInputStream;
Importa ObjectInputStream, que envuelve a FileInputStream para poder reconstruir objetos completos.
import java.io.IOException;
Importa IOException, la excepción base de errores de entrada/salida.
public class RespaldoBinario {
Declara la clase RespaldoBinario.
public static void guardarRespaldoConfig(String configuracion) {
Declara el método static guardarRespaldoConfig, que recibe el texto de configuración a respaldar.
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("config_respaldo.dat"))) {
Abre, con try-with-resources, un ObjectOutputStream que envuelve a un FileOutputStream apuntando a "config_respaldo.dat".

oos.writeObject(configuracion);
Escribe (serializa) el objeto configuracion completo dentro del archivo binario.
System.out.println("Respaldo binario generado correctamente.");
Muestra en consola un mensaje confirmando que el respaldo se generó.
} catch (IOException e) {
Bloque catch que atrapa IOException si algo falla al escribir el archivo binario.
System.out.println("Error al escribir el archivo binario: " + e.getMessage());
Muestra en consola un mensaje de error con el detalle de la excepción.
}
Cierra el bloque catch.
}
Cierra el método guardarRespaldoConfig.
public static void leerRespaldoConfig() {
Declara el método static leerRespaldoConfig, sin parámetros.
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("config_respaldo.dat"))) {
Abre, con try-with-resources, un ObjectInputStream que envuelve a un FileInputStream apuntando al mismo archivo binario.
String configLeida = (String) ois.readObject();
Lee (deserializa) el objeto guardado y lo convierte (cast) de vuelta a String.
System.out.println("Configuración recuperada desde el binario: " + configLeida);
Muestra en consola la configuración recuperada.
} catch (IOException | ClassNotFoundException e) {
Bloque catch que atrapa DOS tipos de excepción a la vez: IOException (error de lectura) o ClassNotFoundException (la clase del objeto guardado ya no existe).
System.out.println("Error al leer el archivo binario: " + e.getMessage());
Muestra en consola un mensaje de error con el detalle de la excepción.
}
Cierra el bloque catch.
}
Cierra el método leerRespaldoConfig.
}
Cierra la clase RespaldoBinario..

Clase de ejecución — PrincipalArchivos.java

Código completo — PrincipalArchivos.java:

```
public class PrincipalArchivos {
    public static void main(String[] args) {
        // Probando Text-File (Trabajo 1)
        HistorialLog.registrarCanje("Ana Torres", "Skin Legendario");
        HistorialLog.leerHistorial();

        // Probando Archivo Binario (Trabajo 2)
        RespaldoBinario.guardarRespaldoConfig("ModoMantto=false;Servidor=Activo");
        RespaldoBinario.leerRespaldoConfig();
    }
}
```

Explicación línea por línea :

public class PrincipalArchivos {
Declara la clase PrincipalArchivos, que probará ambos módulos (texto y binario).
public static void main(String[] args) {
Punto de entrada del programa: aquí comienza la ejecución cuando se corre esta clase.

HistorialLog.registrarCanje("Ana Torres", "Skin Legendario");
Llama a HistorialLog.registrarCanje(...) para escribir una nueva línea en historial_canjes.txt.
HistorialLog.leerHistorial();
Llama a HistorialLog.leerHistorial() para leer y mostrar en consola todo el contenido del archivo de texto.
RespaldoBinario.guardarRespaldoConfig("ModoMantto=false;Servidor=Activo");
Llama a RespaldoBinario.guardarRespaldoConfig(...) para crear/actualizar el archivo binario config_respaldo.dat.
RespaldoBinario.leerRespaldoConfig();
Llama a RespaldoBinario.leerRespaldoConfig() para leer y mostrar en consola el contenido recuperado del archivo binario.
}
Cierra el método main.
}
Cierra la clase PrincipalArchivos.

Salida esperada en consola:

```
Registro de canje guardado exitosamente en el archivo de texto.
--- HISTORIAL DE CANJES (TEXT-FILE) ---
Estudiante: Ana Torres | Ítem: Skin Legendario
Respaldo binario generado correctamente.
Configuración recuperada desde el binario: ModoMantto=false;Servidor=Activo
PS E:\Semestre IV\Taller de lenguajes de programacion\sesion 5>
```

Resumen de Conceptos Clave

- **FileWriter / PrintWriter:** Escriben texto legible en un archivo. PrintWriter facilita escribir línea por línea.
- **FileReader / BufferedReader:** Leen un archivo de texto. BufferedReader permite leer línea por línea de forma eficiente.
- **ObjectOutputStream / ObjectInputStream:** Serializan y deserializan objetos Java completos en archivos binarios.
- **try-with-resources:** Abre un recurso y lo cierra automáticamente al terminar el bloque, incluso si hay error.
- **IOException:** Excepción de entrada/salida; se lanza cuando falla la lectura o escritura de un archivo.
- **Modo append (true):** En FileWriter, el segundo parámetro true hace que se agregue al final del archivo en vez de sobrescribirlo.